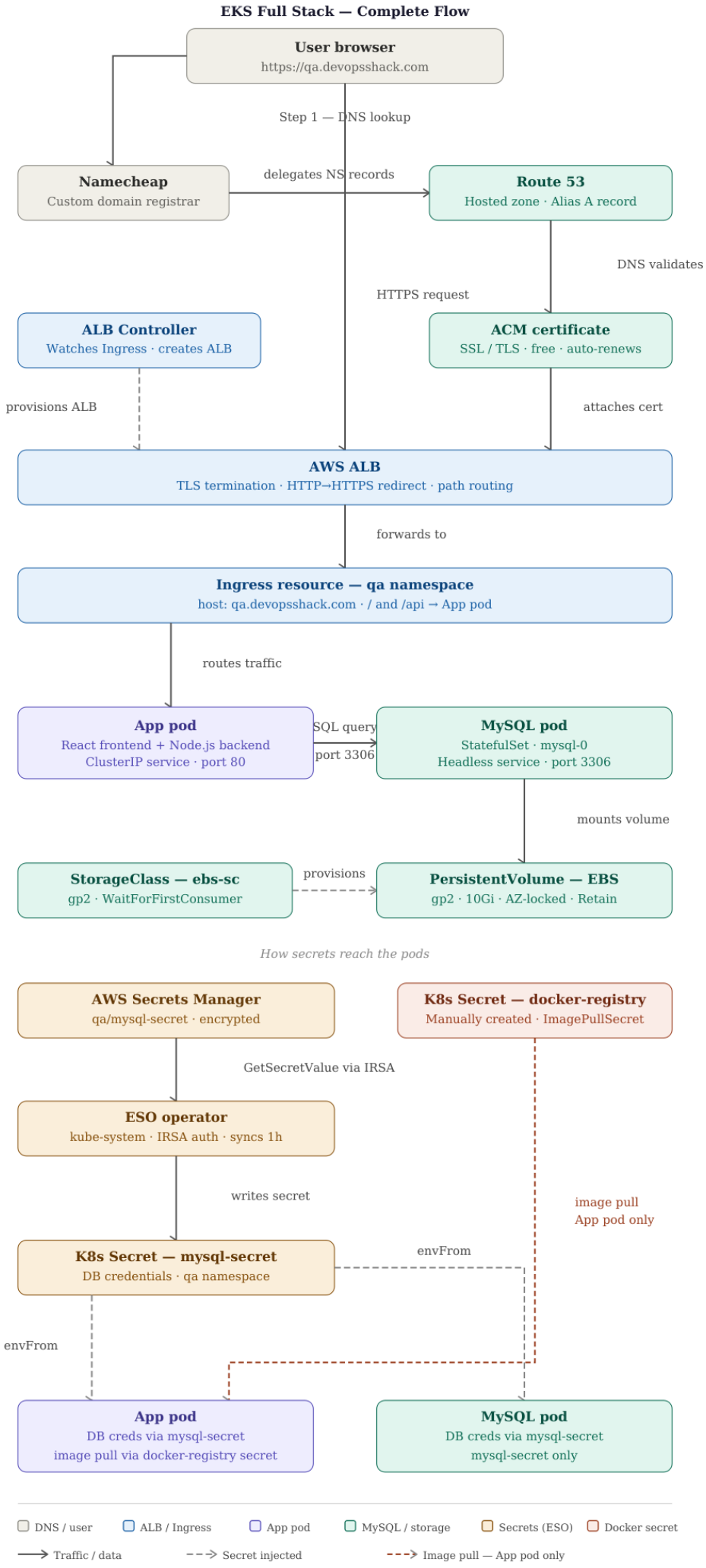

EKS Full Stack — Complete Flow
User browser
https://qa.devopsshack.com
Step 1 — DNS lookup
Namecheap
Custom domain registrar
delegates NS records
Route 53
Hosted zone · Alias A record
DNS validates
HTTPS request
ALB Controller
Watches Ingress · creates ALB
ACM certificate
SSL / TLS · free · auto-renews
provisions ALB
attaches cert
AWS ALB
TLS termination · HTTP→HTTPS redirect · path routing
forwards to
Ingress resource — qa namespace
host: qa.devopsshack.com · / and /api → App pod
routes traffic
App pod
React frontend + Node.js backend
ClusterIP service · port 80
SQL query
port 3306
MySQL pod
StatefulSet · mysql-0
Headless service · port 3306
mounts volume
StorageClass — ebs-sc
gp2 · WaitForFirstConsumer
provisions
PersistentVolume — EBS
gp2 · 10Gi · AZ-locked · Retain
How secrets reach the pods
AWS Secrets Manager
qa/mysql-secret · encrypted
K8s Secret — docker-registry
Manually created · ImagePullSecret
GetSecretValue via IRSA
ESO operator
kube-system · IRSA auth · syncs 1h
writes secret
image pull
App pod only
K8s Secret — mysql-secret
DB credentials · qa namespace
envFrom
envFrom
App pod
DB creds via mysql-secret
image pull via docker-registry secret
MySQL pod
DB creds via mysql-secret
mysql-secret only
DNS / user
ALB / Ingress
App pod
MySQL / storage
Secrets (ESO)
Docker secret
Traffic / data
Secret injected
Image pull — App pod only

Chapter 1

Architecture Overview — How It All Fits Together

1. EKS Full Stack — Architecture Overview

This document provides an exhaustive, production-grade deep-dive into every component, every connection, and every design decision in the EKS Full Stack Complete Flow. The architecture depicted spans DNS resolution, TLS termination, Kubernetes ingress routing, stateful databases, persistent storage, and a fully automated secrets management chain — together forming a secure, scalable, cloud-native 3-tier application running on Amazon EKS.

1.1 The Big Picture

The system consists of two interconnected planes:

- **Traffic Plane: Handles how user requests enter the system, are secured with TLS, routed through the load balancer, and ultimately land in the application pod.**
- **Secrets Plane: Handles how database credentials and registry authentication tokens are created, stored, synced, and injected into pods — without any developer ever hard-coding a secret.**

At a high level, the flow for a user hitting https://qa.devopsshack.com unfolds in exactly 10 distinct stages:

Stage	Component	What Happens
Stage 1	DNS Lookup	Browser queries Namecheap → delegates to Route 53
Stage 2	Route 53 Resolution	Alias A record returns ALB DNS name
Stage 3	TLS Handshake	ACM certificate terminates HTTPS at ALB
Stage 4	ALB Routing	Path-based routing, HTTP→HTTPS redirect
Stage 5	Ingress Resource	Kubernetes Ingress in qa namespace routes /api and /
Stage 6	App Pod	React frontend + Node.js backend in single pod
Stage 7	MySQL Query	App pod sends SQL to mysql-0 via headless service port 3306
Stage 8	PersistentVolume	MySQL mounts EBS gp2 10Gi volume for durable storage

Stage	Component	What Happens
Stage 9	Secret Injection (ESO)	DB creds from AWS Secrets Manager → K8s Secret via IRSA
Stage 10	Image Pull	App pod authenticates to private registry via docker-registry secret

1.2 Technology Stack Summary

Component	Details
Custom Domain	qa.devopsshack.com — registered with Namecheap, NS delegated to Route 53
DNS	Amazon Route 53 — Hosted Zone with Alias A record pointing to ALB
TLS/SSL	AWS Certificate Manager (ACM) — free, auto-renewing certificate
Load Balancer	AWS Application Load Balancer (ALB) — managed by AWS Load Balancer Controller
Ingress	Kubernetes Ingress resource in qa namespace — host + path rules
Application	React frontend (/) + Node.js backend (/api) in one pod on port 80
Database	MySQL 8 StatefulSet — single pod mysql-0, headless service, port 3306
Storage	Amazon EBS gp2 10Gi PersistentVolume — AZ-locked, Retain policy
StorageClass	ebs-sc — gp2 driver, WaitForFirstConsumer binding mode
Secrets (DB)	AWS Secrets Manager → External Secrets Operator (ESO) → K8s Secret
Secrets (Image)	Manually created docker-registry K8s Secret — ImagePullSecret
IRSA	IAM Roles for Service Accounts — zero-credential AWS API auth from pods

Chapter 2
DNS Layer — From Browser to Route 53

2. DNS Layer — Namecheap & Route 53

Before a single byte of application traffic is transmitted, the browser must resolve the hostname qa.devopsshack.com to an IP address or DNS name. This DNS resolution chain is the foundation upon which all subsequent network communication depends.

2.1 Domain Registration — Namecheap

Namecheap is the authoritative domain registrar for devopsshack.com. As a registrar, Namecheap holds the top-level delegation records that tell the internet where to find the authoritative DNS servers for the domain. In this architecture, Namecheap is configured with custom NS (Name Server) records that delegate authority entirely to Amazon Route 53.

2.1.1 NS Delegation

When you create a Hosted Zone in Route 53, AWS assigns four NS records (e.g., ns-1234.awsdns-12.com). These four NS records must be entered verbatim into the Namecheap DNS management panel under Custom DNS. This delegation tells the global DNS resolver hierarchy: "for anything under devopsshack.com, ask Amazon's name servers."

> *Critical: NS records propagate globally through the DNS infrastructure and can take up to 48 hours to fully propagate, though typically this takes only minutes to a few hours with modern resolvers.*

Record	Value
NS Record 1	ns-XXXX.awsdns-XX.com
NS Record 2	ns-XXXX.awsdns-XX.net
NS Record 3	ns-XXXX.awsdns-XX.org
NS Record 4	ns-XXXX.awsdns-XX.co.uk
TTL	172800 seconds (48 hours) — standard for NS records
Purpose	Delegates DNS authority from Namecheap to Route 53

2.2 Amazon Route 53 — Hosted Zone

Amazon Route 53 is a highly available, scalable cloud DNS web service. In this architecture, Route 53 hosts the devopsshack.com Hosted Zone and contains the actual DNS records that map qa.devopsshack.com to the Application Load Balancer.

2.2.1 Alias A Record

The key DNS record for this architecture is an Alias A record for qa.devopsshack.com. Unlike a standard CNAME, an Alias record is a Route 53-specific extension that maps directly to the ALB's DNS name without an additional DNS lookup round-trip. Alias records also work at the zone apex (naked domain), which CNAMEs cannot do.

Attribute	Value
Record Type	A (Alias)
Name	qa.devopsshack.com
Value	Points to AWS ALB DNS name (e.g., k8s-qa-XXXX.ap-south-1.elb.amazonaws.com)
Alias Target	Application Load Balancer in the same AWS region
Health Check	Optional but recommended — Route 53 can monitor ALB health
Routing Policy	Simple routing (single target in this architecture)
TTL	Managed by AWS for Alias records — typically 60 seconds

2.2.2 Why Alias Instead of CNAME?

- No extra DNS lookup: CNAME adds a hop; Alias resolves inline to the IP in the same response
- AWS-native integration: Route 53 automatically updates the Alias target if the ALB changes IPs
- Zone apex support: You can set an Alias at the root devopsshack.com — CNAMEs are forbidden at apex
- No cost for Alias queries to AWS resources in the same region

2.3 DNS Resolution — Step by Step

When a user navigates to https://qa.devopsshack.com, the following DNS resolution chain executes:

- Browser checks local DNS cache — if TTL hasn't expired, uses cached IP directly
- If cache miss: OS forwards query to configured DNS resolver (e.g., 8.8.8.8 or ISP resolver)
- Resolver queries the root DNS servers for .com TLD nameservers
- Root servers return the .com TLD nameservers (Verisign)
- Resolver queries Verisign for devopsshack.com nameservers → returns the Route 53 NS records set by Namecheap

- Resolver queries Route 53 for qa.devopsshack.com → Route 53 returns the ALB DNS name via Alias A record
- Browser receives the ALB IP(s) and initiates TCP connection to port 443

Chapter 3
TLS Termination — ACM & HTTPS Security

3. TLS/SSL — AWS Certificate Manager (ACM)

Transport Layer Security (TLS) ensures that all data transmitted between the user's browser and the ALB is encrypted and authenticated. In this architecture, AWS Certificate Manager (ACM) provides a free, automatically renewing TLS certificate that is attached to the ALB listener.

3.1 How ACM Works

ACM is a managed service that handles the complexity of provisioning, deploying, and renewing SSL/TLS certificates. Certificates issued through ACM for AWS resources are completely free — you only pay for the AWS resources that use them, not the certificate itself.

3.1.1 Certificate Validation — DNS Validation

ACM uses DNS validation to prove you own the domain before issuing a certificate. During validation:

- ACM provides a specific CNAME record (e.g., _abc123.qa.devopsshack.com → _xyz.acm-validations.aws)
- This CNAME is added to the Route 53 Hosted Zone (Route 53 + ACM integration can do this automatically)
- ACM periodically checks for the CNAME's presence — once detected, the certificate is issued
- The validation CNAME record remains in Route 53 permanently — this allows ACM to auto-renew the certificate without any manual intervention

3.1.2 Auto-Renewal

ACM automatically renews managed certificates before expiry (typically 60 days before). As long as the DNS validation record exists in Route 53, renewal is completely hands-free. This eliminates the #1 cause of production outages — expired certificates.

Attribute	Value
Certificate Type	Amazon-issued public certificate
Domain	qa.devopsshack.com (can include *.devopsshack.com wildcard)
Validation Method	DNS validation via Route 53 CNAME record

Attribute	Value
Renewal	Automatic — no human action required
Cost	Free for use with AWS resources
Key Algorithm	RSA 2048 or ECDSA P-256 (AWS default)
Attachment Point	HTTPS listener on the Application Load Balancer

3.2 TLS Termination at the ALB

TLS is terminated at the ALB — this is a deliberate architectural choice with significant implications:

- Offload decryption overhead: The ALB handles all cryptographic operations, so application pods don't waste CPU on TLS
- Centralized certificate management: One certificate at one place — no cert distribution across pods
- HTTP→HTTPS redirect: The ALB can redirect all HTTP (port 80) traffic to HTTPS (port 443) at the load balancer level
- Security: Traffic inside the VPC (ALB → Ingress → Pod) travels over the private network. For highly regulated environments, you can enable end-to-end encryption with pod-level TLS (mTLS), but this adds complexity

> *Production tip: Configure the ALB to use the latest TLS security policy (e.g., ELBSecurityPolicy-TLS13-1-2-2021-06) to disable old TLS 1.0/1.1 and weak ciphers.*

Chapter 4
AWS ALB — Load Balancing, Routing & the Controller

4. AWS Application Load Balancer (ALB)

The AWS Application Load Balancer is the entry point into the Kubernetes cluster for all external HTTPS traffic. It operates at Layer 7 (HTTP/HTTPS) of the OSI model, enabling content-based routing, TLS termination, and health checking at the application level.

4.1 AWS Load Balancer Controller

In a traditional setup, you'd manually provision an ALB in the AWS console and configure it. In Kubernetes on EKS, this entire process is automated by the AWS Load Balancer Controller — an open-source controller from AWS that runs as a Deployment inside the cluster.

4.1.1 What the Controller Does

The AWS Load Balancer Controller constantly watches Kubernetes Ingress resources. When it detects a new Ingress with the annotation kubernetes.io/ingress.class: alb (or ingressClassName: alb), it:

- Calls the AWS EC2 API to provision a new ALB in the same VPC as the EKS cluster
- Creates the necessary Target Groups (one per Kubernetes Service backend)
- Configures Listeners (port 80 for HTTP redirect, port 443 for HTTPS)
- Attaches the ACM certificate to the HTTPS listener
- Registers the EKS node IPs (or pod IPs in IP mode) as targets
- Continuously reconciles — if the Ingress spec changes, the ALB is updated to match

4.1.2 Controller Installation & IRSA

The controller uses IRSA (IAM Roles for Service Accounts) to call AWS APIs without static credentials. It needs an IAM policy with permissions to manage ELB, EC2, ACM, and Route 53 resources. The ServiceAccount in the cluster is annotated with the IAM role ARN, and AWS STS provides temporary credentials via the EKS OIDC provider.

> *The AWS Load Balancer Controller must be installed before creating any Ingress resource — otherwise the Ingress will be created but ignored, and no ALB will be provisioned.*

4.2 ALB Configuration in This Architecture

The ALB created by the controller is configured with the following behavior based on annotations in the Ingress manifest:

Feature	Configuration
Scheme	internet-facing — ALB has a public IP, accessible from the internet
Target Type	ip — routes directly to pod IPs, bypassing kube-proxy (lower latency)
Port 80 (HTTP)	Listener configured to redirect all traffic to port 443 (HTTPS 301)
Port 443 (HTTPS)	Listener configured with ACM certificate, forwards to Ingress target group
Health Check	Checks /health or / on the app pod — ALB only routes to healthy targets
Idle Timeout	Default 60 seconds — configurable via annotation
Access Logs	Can be enabled to S3 for audit and analysis
WAF Integration	Optional — attach AWS WAF for OWASP protection at the ALB level

4.3 Target Group Binding

The ALB routes traffic to one or more Target Groups. Each Target Group maps to a Kubernetes Service. The controller creates the Target Group and registers pod IPs directly (in IP mode) — this means traffic flows: ALB → Pod IP, skipping the NodePort/kube-proxy layer entirely for improved performance.

4.3.1 Health Checks

Target Group health checks ensure ALB only sends traffic to healthy pods. The check is a simple HTTP GET to a configurable path. If a pod fails health checks:

- ALB marks it as unhealthy and stops routing new connections to it
- Kubernetes may simultaneously receive this signal via a Readiness probe failure and mark the pod as NotReady
- Traffic continues to flow to other healthy pods in the same Target Group

Chapter 5
Kubernetes Ingress — Traffic Routing Inside the Cluster

5. Kubernetes Ingress — qa Namespace

The Kubernetes Ingress resource defines the routing rules that tell the ALB how to direct incoming requests to the correct services inside the cluster. In this architecture, the Ingress lives in the qa namespace and defines host-based and path-based routing for the application.

5.1 Ingress Spec — Host and Path Rules

The Ingress is configured with:

- Host: qa.devopsshack.com — ensures this Ingress only handles traffic for this specific hostname
- Path /api → App Service (port 80) — backend API routes
- Path / → App Service (port 80) — React frontend (catch-all)

The complete Ingress manifest looks like this conceptually:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: devopsshack-ingress
  namespace: qa
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:...
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}, {"HTTPS": 443}]'
    alb.ingress.kubernetes.io/ssl-redirect: '443'
spec:
  rules:
  - host: qa.devopsshack.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: app-service
            port:
```

```
            number: 80
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app-service
            port:
              number: 80
```

5.2 Path Routing Logic

With both / and /api pointing to the same service, the routing is actually handled entirely within the Node.js application server — the Ingress simply delivers all traffic to the correct pod. The Node.js backend uses Express routing to handle /api/* requests, while the React SPA handles all other routes client-side.

5.3 Namespace Isolation

The qa namespace provides logical isolation for the QA environment. All resources — the Ingress, App Deployment, MySQL StatefulSet, Services, Secrets, and PVCs — live in the qa namespace. This enables:

- RBAC isolation: Different teams/pipelines can have limited access scoped to qa namespace only
- Resource quotas: Apply CPU/memory limits per namespace to prevent runaway consumption
- Network policies: Restrict which pods can talk to which — e.g., only qa pods can reach the qa MySQL
- Multi-environment support: A prod namespace with identical structure can exist side-by-side

Chapter 6

Application Pod — React + Node.js Backend

6. Application Pod — 3-Tier Frontend & Backend

The application pod is the heart of the system — it runs both the React frontend (served as static assets by Nginx or Node.js itself) and the Node.js/Express backend API. All of this runs as a Kubernetes Deployment in the qa namespace, exposed via a ClusterIP Service on port 80.

6.1 Pod Architecture

Attribute	Value
Image	Private Docker image (e.g., docker.io/devopsshack/app:tag or ECR URI)
Containers	Single container running Node.js (backend) + serves React build (frontend)
Port	80 — internal ClusterIP service target port
Service Type	ClusterIP — only accessible within the cluster, not externally
Replicas	Configurable — typically 2+ for HA in production, 1 in QA
Resource Limits	CPU/Memory requests and limits defined to enable proper scheduling
Liveness Probe	HTTP GET / — restart pod if it becomes unresponsive
Readiness Probe	HTTP GET / — remove from service endpoints if not ready to serve

6.2 How the App Pod Gets Its Configuration

The application needs database credentials to connect to MySQL. These are injected as environment variables through envFrom — a Kubernetes mechanism that loads all keys from a Secret as environment variables into the pod. The App pod uses two secret references:

- mysql-secret (from ESO): Provides DB_HOST, DB_USER, DB_PASSWORD, DB_NAME — the MySQL connection credentials
- docker-registry secret: Used as an imagePullSecret in the pod spec — tells Kubernetes how to authenticate to the private Docker registry to pull the container image

```
env:
```

```
  envFrom:
  - secretRef:
      name: mysql-secret
imagePullSecrets:
- name: docker-registry
```

6.3 React Frontend Serving

In a typical Node.js 3-tier setup, the React application is built (npm run build) and the resulting static files (index.html, JS bundles, CSS) are either:

- Served directly by Express using express.static() pointing at the build/ directory
- Or served by an Nginx sidecar container in the same pod

All React routes (client-side routing) require a catch-all — Express returns index.html for any non-/api route, allowing React Router to handle client-side navigation.

6.4 MySQL Communication

The app pod communicates with MySQL via the headless service DNS name: mysql-0.mysql.qa.svc.cluster.local on port 3306. The connection is typically established using mysql2 or Sequelize in the Node.js backend. The credentials are loaded from environment variables injected by the mysql-secret Secret.

> *Best practice: Use connection pooling (e.g., mysql2 createPool) to manage MySQL connections efficiently. A single Node.js instance handling many concurrent requests should maintain 5-20 pooled connections.*

Chapter 7

MySQL Pod — StatefulSet, Headless Service & Persistence

7. MySQL — StatefulSet Architecture

MySQL is deployed as a Kubernetes StatefulSet — not a Deployment. This is a critical distinction. StatefulSets provide stable pod identity, stable network hostnames, and ordered deployment/scaling — all of which are essential for stateful database workloads.

7.1 Why StatefulSet, Not Deployment?

Feature	Why It Matters for MySQL
Stable Pod Name	mysql-0 — always mysql-0, even after restart (Deployment pods get random names)
Stable Network Identity	DNS: mysql-0.mysql.qa.svc.cluster.local — predictable, always consistent
Ordered Operations	Pods start/stop in order (mysql-0 before mysql-1) — critical for replication setup
PVC per Pod	Each replica gets its own PVC — data doesn't get swapped between replicas
Sticky Volume	mysql-0 always gets its same PVC, even if rescheduled to a different node

7.2 Headless Service

MySQL is exposed inside the cluster via a headless service — a Service with clusterIP: None. Unlike a normal ClusterIP Service that provides a virtual IP and load balances across pods, a headless service has no virtual IP. Instead, DNS for the headless service returns the individual pod IPs directly.

The headless service is named mysql in the qa namespace. Its DNS behavior:

- mysql.qa.svc.cluster.local → returns all pod IPs (for discovery/replication in multi-replica setups)
- mysql-0.mysql.qa.svc.cluster.local → returns the IP of pod mysql-0 specifically

The App pod uses mysql-0.mysql.qa.svc.cluster.local as the DB_HOST — connecting directly and predictably to the single primary MySQL instance.

```
apiVersion: v1
```

```
kind: Service
metadata:
  name: mysql
  namespace: qa
spec:
  clusterIP: None   # This makes it headless
  selector:
    app: mysql
  ports:
  - port: 3306
    targetPort: 3306
```

7.3 StatefulSet Spec

The MySQL StatefulSet (mysql-0) defines a volumeClaimTemplate that automatically creates a PersistentVolumeClaim for each pod replica. This PVC binds to a PersistentVolume backed by an EBS volume:

Attribute	Value
StatefulSet Name	mysql
Pod Name	mysql-0 (first/only replica)
Replicas	1 (single primary — no replication in this setup)
Container Image	mysql:8.0 or similar
Port	3306
Volume Mount	/var/lib/mysql — standard MySQL data directory
VolumeClaimTemplate	Requests 10Gi from ebs-sc StorageClass
Environment	MYSQL_ROOT_PASSWORD, MYSQL_DATABASE from mysql-secret via envFrom

Chapter 8
Persistent Storage — EBS, StorageClass & PersistentVolume

8. Persistent Storage — EBS & Kubernetes

For a stateful database like MySQL, persistent storage is non-negotiable. Kubernetes uses a three-layer storage abstraction: StorageClass → PersistentVolumeClaim → PersistentVolume. The underlying storage in this architecture is Amazon EBS (Elastic Block Store).

8.1 StorageClass — ebs-sc

The StorageClass named ebs-sc defines the type and behavior of storage that Kubernetes can dynamically provision. When a PVC requests storage from ebs-sc, the EBS CSI Driver automatically creates an EBS volume in AWS and binds it as a PersistentVolume.

Parameter	Value
StorageClass Name	ebs-sc
Provisioner	ebs.csi.aws.com — EBS CSI Driver (must be installed in cluster)
Volume Type	gp2 — General Purpose SSD, 3 IOPS/GB baseline, burst to 3000
Binding Mode	WaitForFirstConsumer — critical for AZ affinity (see below)
Reclaim Policy	Retain — PV is NOT deleted when PVC is deleted
Volume Expansion	Enabled — PVC size can be increased (but not decreased)
Encryption	Can be enabled with KMS key ID in StorageClass parameters

8.1.1 WaitForFirstConsumer Binding Mode

This is one of the most important storage configuration decisions. With the default Immediate binding mode, Kubernetes creates the EBS volume in a random Availability Zone as soon as the PVC is created — potentially in a different AZ than where the MySQL pod will be scheduled. Since EBS volumes are AZ-locked (you cannot attach an EBS volume to an EC2 instance in a different AZ), this would cause the pod to fail to start.

WaitForFirstConsumer delays PV provisioning until a Pod is actually scheduled to a node. At that point, Kubernetes knows which node (and therefore which AZ) the pod will run in, and provisions the EBS volume in the correct AZ. This eliminates AZ mismatch errors entirely.

> *AZ-locking is the #1 gotcha with EBS in Kubernetes. WaitForFirstConsumer is the correct solution and should always be used for EBS-backed StorageClasses on EKS.*

8.2 PersistentVolume — gp2 10Gi

Once the MySQL pod is scheduled, the EBS CSI Driver:

- Calls the AWS EC2 API to create a 10Gi gp2 EBS volume in the correct AZ
- Creates a PersistentVolume object in Kubernetes representing that EBS volume
- Binds the PVC (created by the StatefulSet volumeClaimTemplate) to the PV
- Attaches the EBS volume to the EC2 node where mysql-0 is scheduled
- Mounts the EBS volume into the pod at /var/lib/mysql

Attribute	Value
Capacity	10Gi
Volume Type	gp2 (AWS EBS General Purpose SSD)
IOPS	30 IOPS baseline (3 IOPS/GB × 10GB), burstable to 3000
AZ	Locked to the AZ of the EC2 node running mysql-0
Reclaim Policy	Retain — even after PVC deletion, the EBS volume persists
Access Mode	ReadWriteOnce — only one node can mount in read-write mode at a time
Mount Path	/var/lib/mysql — all MySQL data files, InnoDB tablespaces, logs

8.3 Data Durability Considerations

EBS gp2 volumes provide 99.8%–99.9% annual durability. For production MySQL:

- Enable automated RDS-style backups using Velero + AWS EBS snapshots — takes point-in-time snapshots
- Consider migrating from gp2 to gp3 — gp3 provides 3000 IOPS baseline at lower cost and doesn't require burst credits
- For production workloads, Amazon RDS or Aurora MySQL with Multi-AZ is strongly preferred over self-managed MySQL on EKS

Chapter 9
Secrets Management — ESO, IRSA & AWS Secrets Manager

9. Secrets Management Architecture

This is the most sophisticated part of the architecture. The secrets management chain ensures that database credentials are stored securely in AWS Secrets Manager, synced automatically into Kubernetes Secrets by the External Secrets Operator (ESO) using IRSA for zero-credential AWS API access, and injected into pods via envFrom.

9.1 The Problem — Why Not Just kubectl create secret?

The naive approach to Kubernetes secrets is to run kubectl create secret generic mysql-secret --from-literal=DB_PASSWORD=mypassword. This has several serious problems:

- Secrets in etcd are only base64-encoded, not encrypted, by default — anyone with kubectl access can read them
- Secrets get checked into Git accidentally (the pipeline needs to create them somehow)
- No rotation: changing a password requires manual kubectl intervention
- No audit trail: no record of who accessed the secret when

The ESO + AWS Secrets Manager approach solves all of these problems.

9.2 AWS Secrets Manager

AWS Secrets Manager is a managed secrets store with:

- Encryption at rest using AWS KMS (AES-256)
- Fine-grained IAM access control — only authorized IAM roles/users can read the secret
- Automatic rotation — can rotate MySQL passwords automatically on a schedule
- Full audit trail via AWS CloudTrail — every GetSecretValue call is logged
- Versioning — previous secret versions are retained for rollback

Attribute	Value
Secret Path	qa/mysql-secret (namespace-prefixed naming convention)
Secret Format	JSON: {"DB_HOST": "...", "DB_USER": "...", "DB_PASSWORD": "...", "DB_NAME": "..."}

Attribute	Value
Encryption	AES-256 via AWS KMS — encrypted at rest
Access Control	IAM policy restricts access to specific IAM role (IRSA role for ESO)
Rotation	Optional — can auto-rotate MySQL password and update the secret value
Audit	Every read logged in CloudTrail — who, when, from where

9.3 IRSA — IAM Roles for Service Accounts

IRSA is the mechanism that allows the ESO pod running inside EKS to call the AWS Secrets Manager API without using static IAM user credentials. It works via OIDC federation between the EKS cluster and AWS IAM.

9.3.1 IRSA — How It Works

- EKS creates an OIDC provider URL (e.g., oidc.eks.ap-south-1.amazonaws.com/id/XXXX)
- An IAM role is created with a trust policy that allows the OIDC provider to assume it — scoped to a specific namespace and service account
- The ESO Kubernetes ServiceAccount is annotated with the IAM role ARN
- When the ESO pod starts, the EKS Pod Identity Agent (or the projected service account token) presents a signed JWT to AWS STS
- AWS STS validates the JWT against the OIDC provider and returns temporary credentials (AccessKeyId, SecretAccessKey, SessionToken) valid for 1 hour
- ESO uses these temporary credentials to call secretsmanager:GetSecretValue
- No static credentials, no IAM user keys, no secrets in environment variables — zero-trust by design

> *IRSA is the gold standard for AWS API access from EKS pods. Never use static IAM credentials (aws_access_key_id/aws_secret_access_key) in pod environment variables or configmaps.*

9.4 External Secrets Operator (ESO)

The External Secrets Operator is a Kubernetes controller (installed in kube-system namespace) that bridges the gap between AWS Secrets Manager and Kubernetes Secrets. It is configured with two CRDs: SecretStore (defines how to authenticate to the secret backend) and ExternalSecret (defines which secrets to sync and how to map them).

9.4.1 SecretStore

```
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: aws-secrets-manager
  namespace: qa
spec:
  provider:
    aws:
      service: SecretsManager
      region: ap-south-1
      auth:
        jwt:
          serviceAccountRef:
            name: external-secrets-sa  # Has IRSA annotation
```

9.4.2 ExternalSecret

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: mysql-external-secret
  namespace: qa
spec:
  refreshInterval: 1h   # Syncs every 1 hour
  secretStoreRef:
    name: aws-secrets-manager
    kind: SecretStore
  target:
    name: mysql-secret   # Name of K8s Secret to create/update
    creationPolicy: Owner
  dataFrom:
  - extract:
      key: qa/mysql-secret
```

9.4.3 The Sync Cycle

ESO runs a sync every 1 hour (refreshInterval: 1h). In each cycle:

- ESO calls secretsmanager:GetSecretValue for qa/mysql-secret
- AWS STS validates IRSA credentials, returns temp creds
- ESO receives the JSON secret value
- ESO creates or updates the K8s Secret named mysql-secret in the qa namespace
- The Secret's data fields are the base64-encoded JSON key-value pairs
- Pods using envFrom: secretRef: mysql-secret automatically receive updated env vars on next restart

9.5 K8s Secret — docker-registry

The second secret in the architecture is the docker-registry ImagePullSecret. This is a manually created Secret of type kubernetes.io/dockerconfigjson. It contains credentials for authenticating to the private Docker registry to pull the application container image.

```
kubectl create secret docker-registry docker-registry \
  --docker-server=docker.io \
  --docker-username=devopsshack \
  --docker-password=<PAT or password> \
  --namespace=qa
```

This secret is only needed by the App pod (not MySQL, which uses a public image). It is referenced in the Deployment spec as imagePullSecrets. MySQL pods pull from a public registry and don't need this secret.

Chapter 10

End-to-End Request & Secret Flow — Complete Walkthrough

10. End-to-End Flow Walkthrough

10.1 Traffic Flow — User Request to Database Query

Here is the exact sequence of events when a user submits a form on the React frontend that triggers a database write:

Step	Layer	What Happens
1	Browser DNS	Browser queries for qa.devopsshack.com → Namecheap NS → Route 53 → Alias A record → ALB DNS name → ALB IP
2	TCP + TLS	Browser opens TCP connection to ALB:443. TLS handshake — ACM certificate presented, browser validates. Encrypted channel established
3	HTTPS Request	Browser sends HTTPS POST /api/items with JSON body inside encrypted TLS tunnel
4	ALB Receives	ALB terminates TLS, decrypts the request. Checks listener rules — HTTPS:443 → forward to Target Group
5	Target Group	ALB selects a healthy pod IP from the Target Group (registered by ALB Controller). Forwards HTTP request to pod:80
6	Node.js Backend	Express receives POST /api/items. Validates request. Constructs SQL INSERT query
7	MySQL Connection	Node.js uses connection pool to send SQL to mysql-0.mysql.qa.svc.cluster.local:3306
8	CoreDNS	Kubernetes CoreDNS resolves mysql-0.mysql.qa.svc.cluster.local to the mysql-0 pod IP (via headless service)
9	MySQL Executes	mysql-0 pod receives SQL query. InnoDB writes data to /var/lib/mysql on the EBS-backed volume
10	EBS Write	EBS gp2 volume (gp2 IOPS) physically writes the data. Returns success to MySQL
11	Response Chain	MySQL → Node.js → ALB → TLS encrypt → Browser. User sees success confirmation

10.2 Secrets Flow — From AWS to Pod Environment

This flow happens in the background, independently of user requests, on a scheduled basis:

Step	Stage	Detail
1	ESO Sync Trigger	ESO reconcile loop fires every 1 hour (or on ExternalSecret creation/update)
2	IRSA Token Request	ESO's ServiceAccount has projected token volume. ESO pod reads the JWT token from /var/run/secrets
3	STS Exchange	ESO sends JWT + role ARN to AWS STS AssumeRoleWithWebIdentity. STS validates JWT signature against OIDC provider
4	Temp Credentials	STS returns time-limited AccessKeyId, SecretAccessKey, SessionToken (valid 1 hour)
5	Secrets Manager Call	ESO calls secretsmanager:GetSecretValue('qa/mysql-secret') with temp credentials
6	Secret Received	AWS returns JSON: {DB_HOST, DB_USER, DB_PASSWORD, DB_NAME}
7	K8s Secret Written	ESO creates/patches mysql-secret in qa namespace with the decoded key-value pairs
8	Pod Consumes	New/restarted pods have these values as env vars via envFrom: secretRef: mysql-secret

Chapter 11
Troubleshooting Guide — Common Issues & Fixes

11. Troubleshooting Common Issues

11.1 DNS & ALB Issues

Symptom	Diagnosis & Fix
Site not resolving	Check NS records in Namecheap match Route 53 hosted zone NS records exactly. Use: dig +trace qa.devopsshack.com
ERR_SSL_PROTOCOL_ERROR	ACM certificate not attached to ALB HTTPS listener. Check Ingress annotation: alb.ingress.kubernetes.io/certificate-arn
ALB not created	kubectl describe ingress -n qa — check events. ALB Controller pod may be crashing. Check IRSA role has correct permissions
HTTP 502 Bad Gateway	ALB health check failing. Pod not ready. Check: kubectl get pods -n qa, check Readiness probe config
HTTP 301 redirect loop	SSL redirect annotation misconfigured. Ensure ssl-redirect: '443' annotation is set

11.2 MySQL & Storage Issues

Symptom	Diagnosis & Fix
mysql-0 pod stuck Pending	PVC not bound. Check: kubectl get pvc -n qa. If Pending, the EBS CSI Driver may not be installed or IRSA for CSI driver is missing
EBS volume in wrong AZ	StorageClass is using Immediate binding mode. Change to WaitForFirstConsumer and recreate PVC
MySQL container CrashLoopBackOff	Check: kubectl logs mysql-0 -n qa. Common causes: wrong root password, data directory corruption, permission issue on /var/lib/mysql
Can't connect to MySQL from app pod	Test: kubectl exec -it <app-pod> -n qa -- nc -zv mysql-0.mysql.qa.svc.cluster.local 3306. Check NetworkPolicy if one exists

11.3 Secrets & ESO Issues

Symptom	Diagnosis & Fix
mysql-secret not created	kubectl describe externalsecret mysql-external-secret -n qa. Check .status.conditions for error message
ESO SecretSyncedError	IRSA credentials may have expired or role ARN is wrong. Verify ServiceAccount annotation and IAM trust policy
App pod env var empty	Restart pod after Secret is populated: kubectl rollout restart deployment/app -n qa
Image pull error (ErrImagePull)	docker-registry secret missing or wrong credentials. Recreate with: kubectl create secret docker-registry...
ESO not syncing	Check ESO pod is running: kubectl get pods -n kube-system | grep external-secrets. Check logs for auth errors

Chapter 12
Security Best Practices & Production Hardening

12. Security Best Practices

12.1 Network Security

- Enable VPC Security Groups on EKS nodes — restrict inbound to ALB security group only
- Use Kubernetes NetworkPolicies to restrict pod-to-pod communication — only app pods should reach MySQL on 3306
- Enable VPC Flow Logs for network audit and anomaly detection
- Keep the EKS API server endpoint private or restrict with CIDR whitelist
  - eks.amazonaws.com endpoint: set publicAccessCidrs to your office/VPN IP ranges

12.2 Secrets Security

- Enable EKS Secrets encryption using KMS — encrypts etcd at rest for K8s Secrets
- Use secretsmanager:GetSecretValue with resource-level ARN conditions in IAM policies
- Rotate the docker-registry PAT regularly — use a bot account, not personal credentials
- Consider using ECR (Elastic Container Registry) instead of Docker Hub — ECR integrates natively with EKS via IRSA, no ImagePullSecret needed

12.3 Container Security

- Run containers as non-root — set runAsNonRoot: true and runAsUser: 1000 in securityContext
- Use read-only root filesystem — readOnlyRootFilesystem: true (except /var/lib/mysql for MySQL)
- Scan container images with Trivy in CI pipeline before pushing to registry
- Set CPU and memory limits on all containers — prevents resource exhaustion attacks
- Use Pod Security Standards (Restricted profile) for application pods

12.4 EBS Security

- Enable EBS volume encryption using KMS — configure in the StorageClass parameters
- Use EBS snapshots + AWS Backup for automated point-in-time recovery
- Restrict EBS CSI Driver IRSA role to specific volume ARNs where possible

Appendix
Glossary of Key Terms

Appendix: Glossary

Term	Definition
ALB	Application Load Balancer — AWS L7 load balancer with content-based routing
ACM	AWS Certificate Manager — managed TLS/SSL certificate service
CNAME	Canonical Name record — DNS record that maps one hostname to another
CoreDNS	Kubernetes' internal DNS server — resolves service and pod hostnames within the cluster
CSI	Container Storage Interface — standard API for storage plugins in Kubernetes
EBS	Elastic Block Store — AWS block storage, equivalent to a virtual hard drive for EC2/EKS
ESO	External Secrets Operator — Kubernetes controller that syncs secrets from external stores into K8s Secrets
IRSA	IAM Roles for Service Accounts — mechanism for K8s pods to assume AWS IAM roles via OIDC
KMS	Key Management Service — AWS managed encryption key service
NS Record	Name Server record — DNS record that delegates a zone to specific name servers
OIDC	OpenID Connect — identity protocol used by EKS for IRSA federation with AWS IAM
PV	PersistentVolume — a piece of storage in the cluster provisioned by an admin or dynamically
PVC	PersistentVolumeClaim — a request for storage by a Pod
StatefulSet	Kubernetes workload for stateful applications — provides stable identities and ordered operations
STS	Security Token Service — AWS service that issues temporary credentials via AssumeRole
TLS	Transport Layer Security — cryptographic protocol for secure communication over networks
VPC	Virtual Private Cloud — isolated AWS network environment for your resources

Document prepared by Aditya Jaiswal · DevOps Shack

devopsshack.com · @devopsshack on YouTube, LinkedIn & Instagram